



UTS DEEP LEARNING (PROJEK)



CONVOLUTIONAL NEURAL NETWORK CIFAR-10 DATASET

Kelompok 6:

Rio Tri Wibowo
112209400000004

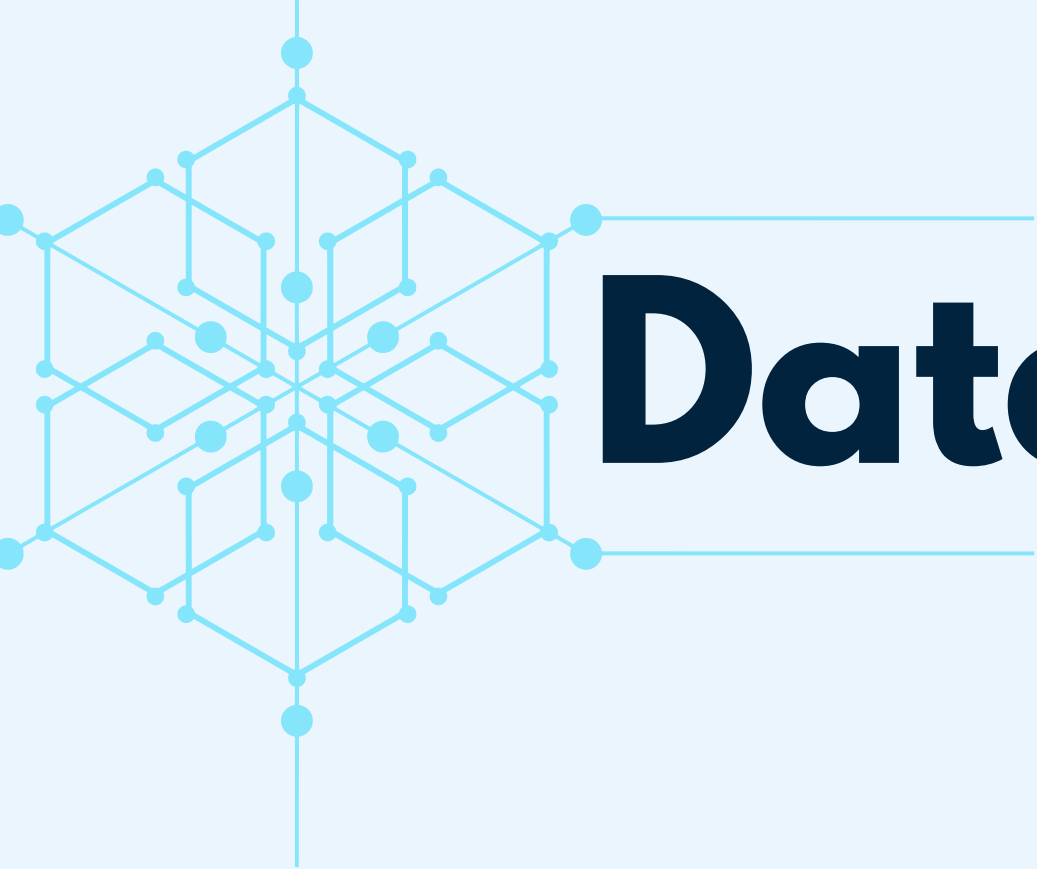
Aisya Shalifa
112209400000032



Goals

- Klasifikasi 10 kelas dari dataset CIFAR-10
- Melakukan optimasi hyperparameter untuk meningkatkan akurasi
- Melakukan analisis feature untuk melihat kesalahan prediksi dari model
- Membandingkan arsitektur CNN dengan ResNet





Dataset

Terdiri dari 10 kelas, yaitu:
plane, car, bird, cat, deer, dog,
frog, horse, ship, truck.

Kumpulan data CIFAR-10 terdiri
dari 60.000 gambar berwarna
berukuran 32x32 piksel.

Total: 60.000 gambar
Training: 50.000 gambar (90%)
Uji: 10.000 gambar (10%)

The background is a solid dark blue. In the top-left corner, there is a faint, light blue geometric pattern of interconnected lines and dots, resembling a network or a molecular structure. On the right side, there are several abstract, glossy, blue and white spheres and rings, some of which appear to be part of a larger, flowing structure. The main text 'CNN' is centered on the left side.

CNN



Arsitektur



```
[ ] # Define custom CNN model
class CustomCNN(nn.Module):
    def __init__(self):
        super(CustomCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.bn1 = nn.BatchNorm2d(32)
        self.bn2 = nn.BatchNorm2d(64)
        self.bn3 = nn.BatchNorm2d(128)
        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.fc2 = nn.Linear(256, 10)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.pool(F.relu(self.bn1(self.conv1(x))))
        x = self.pool(F.relu(self.bn2(self.conv2(x))))
        x = self.pool(F.relu(self.bn3(self.conv3(x))))
        x = x.view(-1, 128 * 4 * 4)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```



Hyperparameter



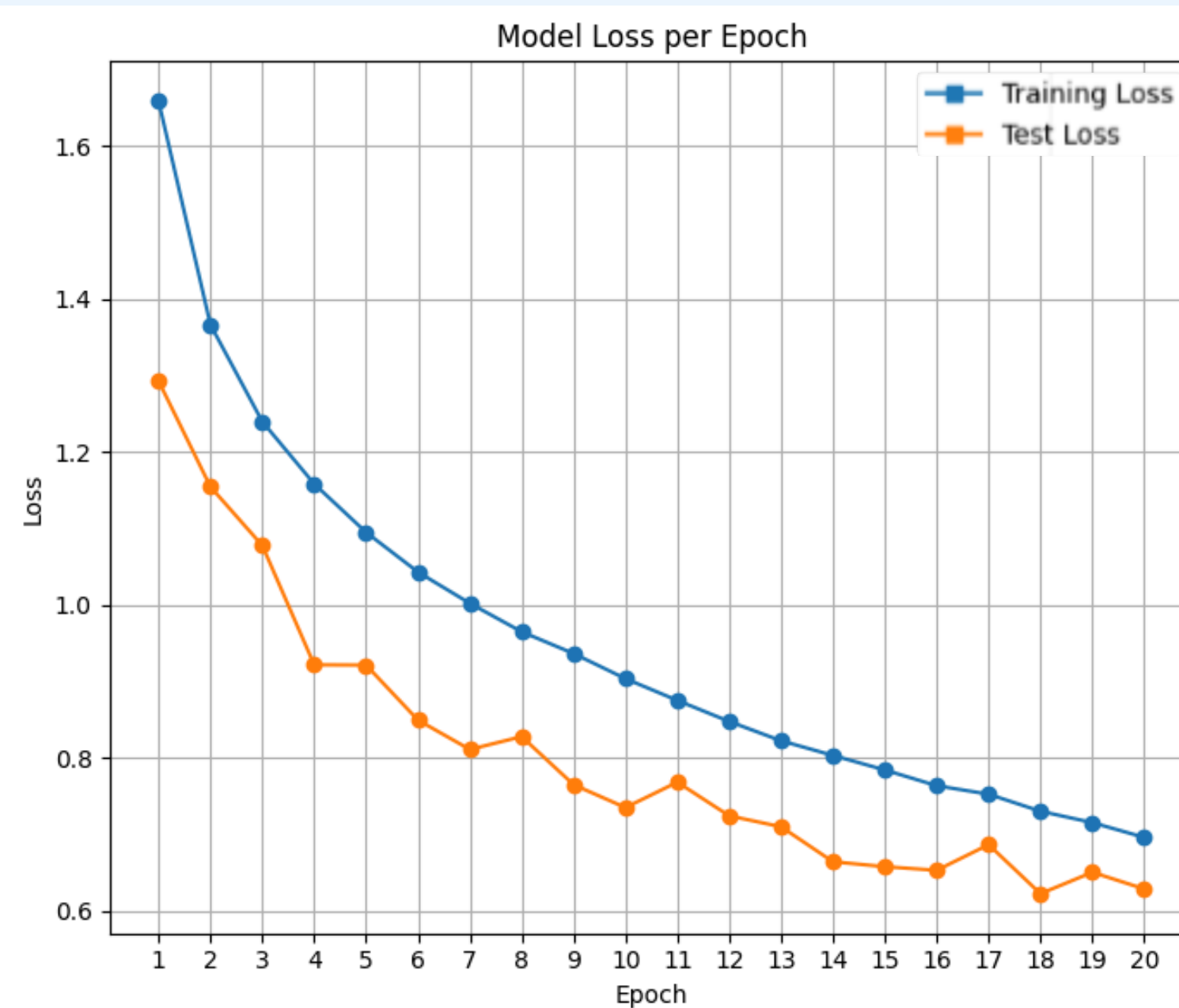
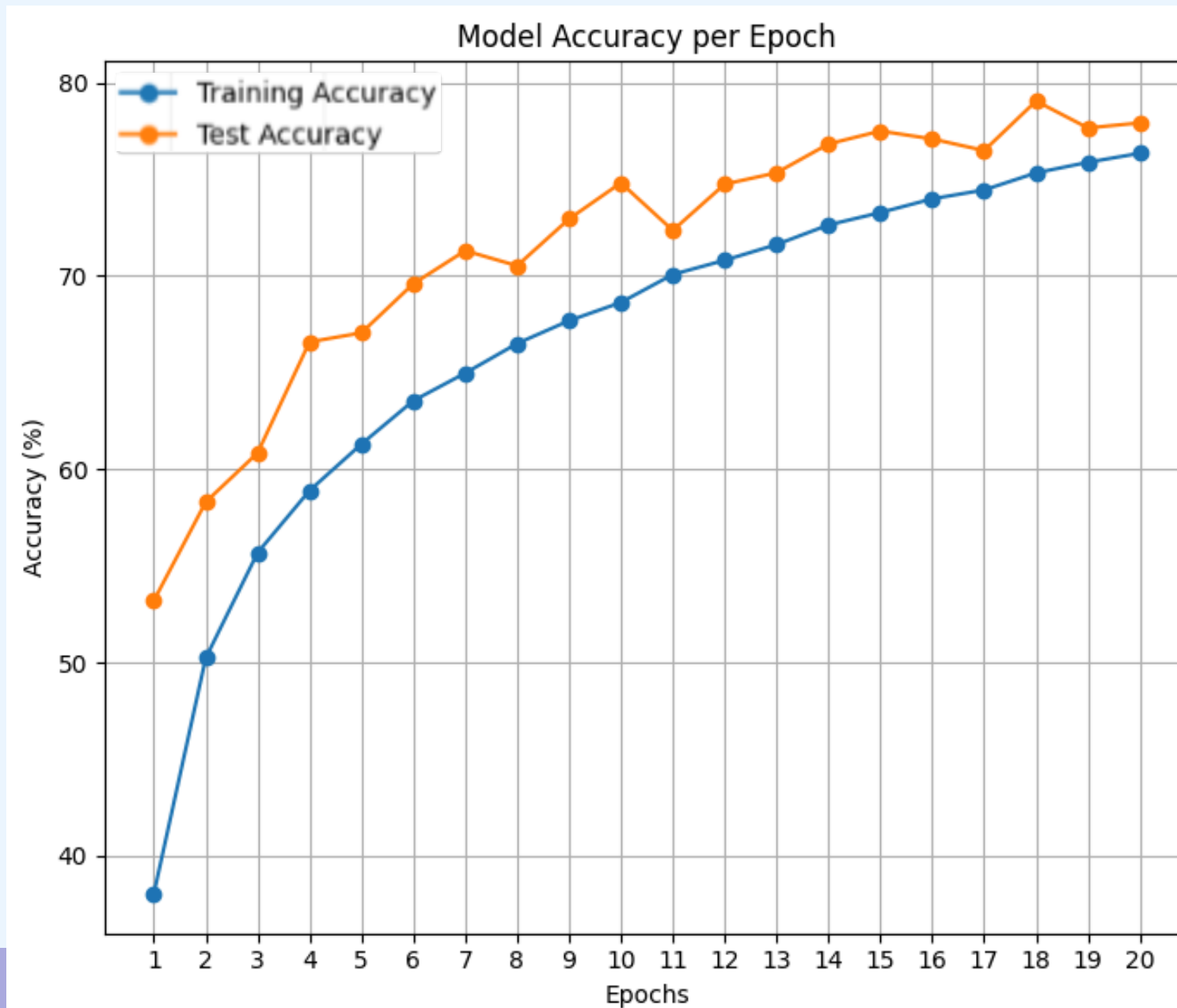
- Learning rate (lr): [0.001, 0.003]
- Batch size:[32, 64]
- Optimizer: Adam
- Epochs: 10
- Patience: 3

Hasil

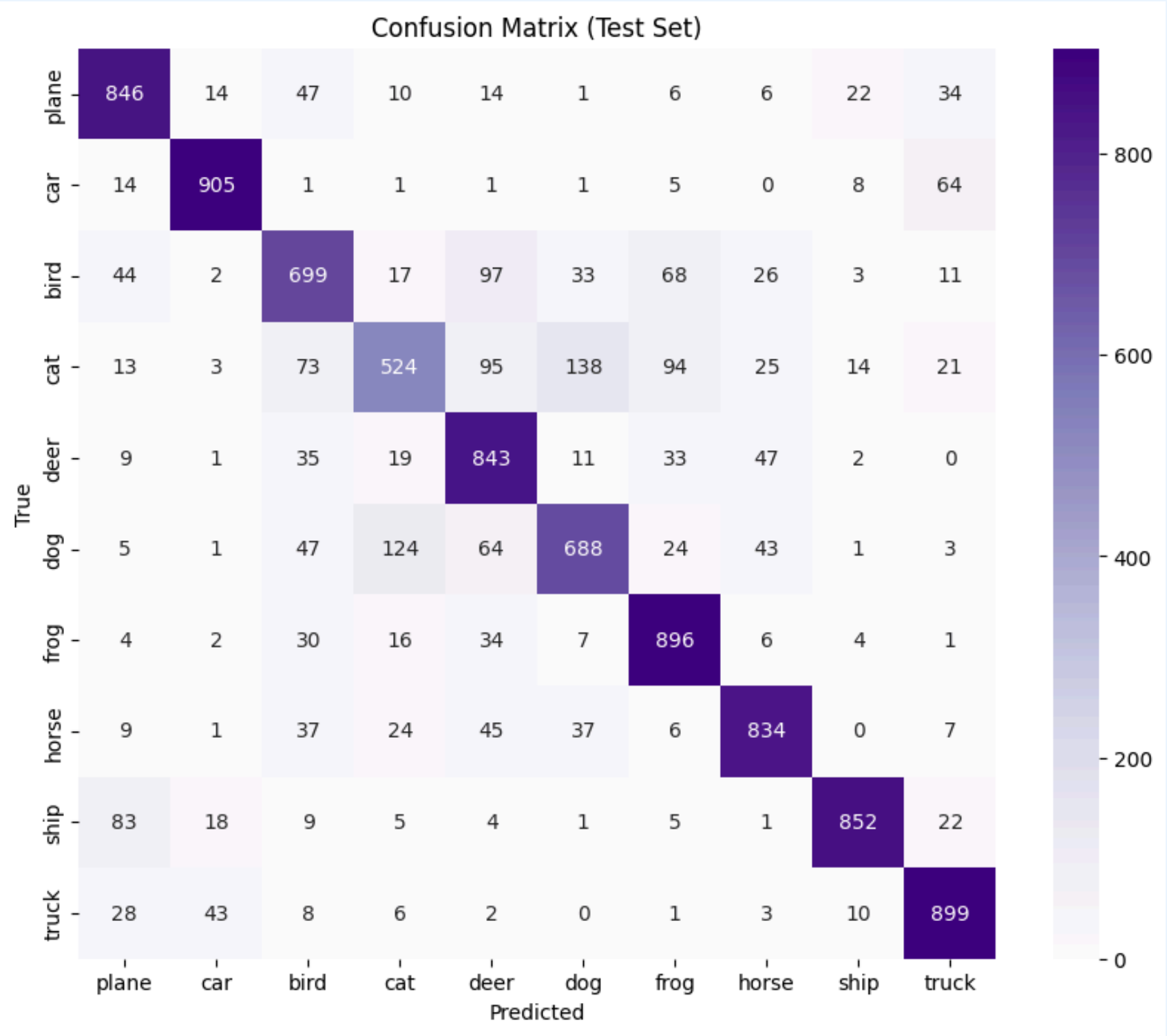
- lr terbaik: 0.001
- Batch size terbaik: 32
- Akurasi 76.41%



Grafik Loss dan Akurasi



Confusion Matrix



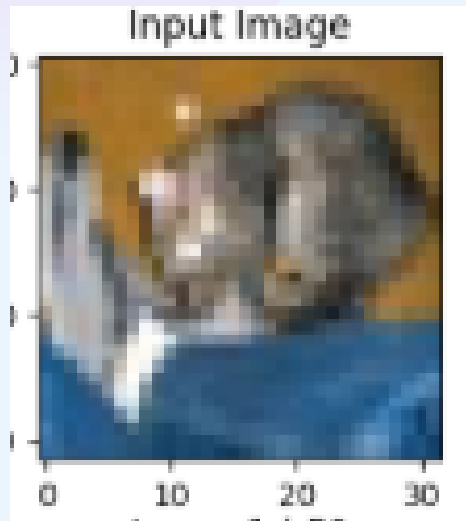
Laporan Klasifikasi Kelas



Per-Class Metrics (Test Set):

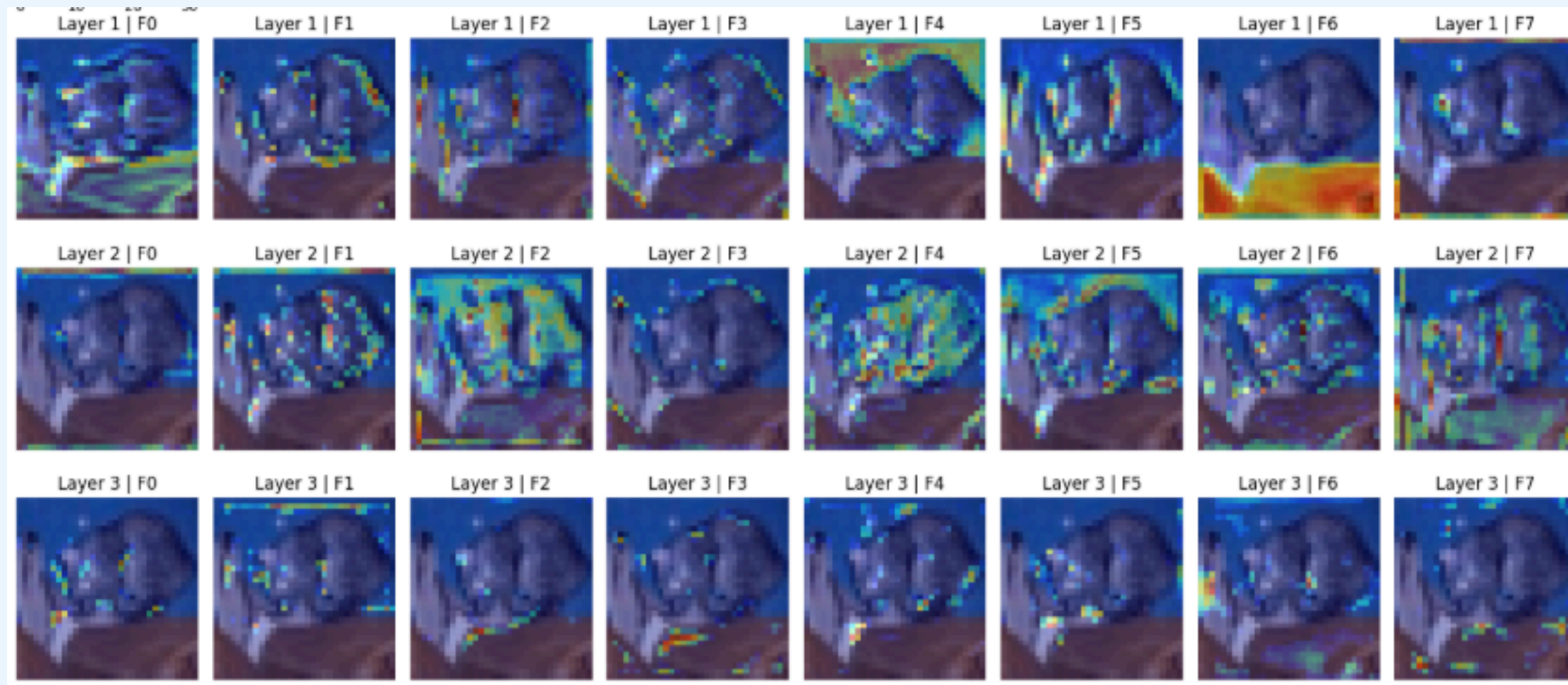
plane	: Precision: 0.802, Recall: 0.846, F1-Score: 0.823
car	: Precision: 0.914, Recall: 0.905, F1-Score: 0.910
bird	: Precision: 0.709, Recall: 0.699, F1-Score: 0.704
cat	: Precision: 0.702, Recall: 0.524, F1-Score: 0.600
deer	: Precision: 0.703, Recall: 0.843, F1-Score: 0.767
dog	: Precision: 0.750, Recall: 0.688, F1-Score: 0.718
frog	: Precision: 0.787, Recall: 0.896, F1-Score: 0.838
horse	: Precision: 0.842, Recall: 0.834, F1-Score: 0.838
ship	: Precision: 0.930, Recall: 0.852, F1-Score: 0.889
truck	: Precision: 0.847, Recall: 0.899, F1-Score: 0.872

Feature Maps



Pred: Cat
True: Cat

1. Prediksi model benar
 - Model memprediksi gambar sebagai “Cat”, dan label aslinya juga “Cat”.
2. Aktivasi layer awal menangkap fitur sederhana
 - Seperti tepi, warna, dan tekstur dari gambar.
3. Aktivasi layer dalam menangkap fitur kompleks
 - Seperti bentuk wajah kucing, pola keseluruhan, dan struktur semantik.
4. Model fokus pada bagian penting dari objek
 - Aktivasi lebih kuat pada area kucing, bukan latar belakang → menandakan model belajar fokus ke objek utama.
5. Pemrosesan bertingkat terlihat jelas
 - Informasi ditransformasikan secara bertahap dari fitur rendah ke tinggi melalui layer-layer CNN.



RESNET



Arsitektur



```
# Helper functions (sudah benar dari kode Anda)
def conv3x3(in_planes, out_planes, stride=1):
    """3x3 convolution with padding"""
    return nn.Conv2d(in_planes, out_planes, kernel_size=3, stride=stride,
                     padding=1, bias=False)

def conv1x1(in_planes, out_planes, stride=1):
    """1x1 convolution"""
    return nn.Conv2d(in_planes, out_planes, kernel_size=1, stride=stride, bias=False)

class Bottleneck(nn.Module):
    expansion = 4 # Faktor ekspansi untuk output channels dari bottleneck block

    def __init__(self, inplanes, planes, stride=1, downsample=None):
        super(Bottleneck, self).__init__()
        # conv1: 1x1 convolution untuk mereduksi dimensi (jika planes < inplanes) atau mengubahnya
        self.conv1 = conv1x1(inplanes, planes)
        self.bn1 = nn.BatchNorm2d(planes)
        # conv2: 3x3 convolution utama
        self.conv2 = conv3x3(planes, planes, stride) # Stride diterapkan di sini untuk downsampling spasial
        self.bn2 = nn.BatchNorm2d(planes)
        # conv3: 1x1 convolution untuk mengekspansi dimensi kembali
        self.conv3 = conv1x1(planes, planes * self.expansion)
        self.bn3 = nn.BatchNorm2d(planes * self.expansion)
        self.relu = nn.ReLU(inplace=True)
        self.downsample = downsample # Untuk shortcut connection jika dimensi tidak cocok
        self.stride = stride
```

```
def forward(self, x):
    identity = x

    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)

    out = self.conv2(out)
    out = self.bn2(out)
    out = self.relu(out)

    out = self.conv3(out)
    out = self.bn3(out)

    if self.downsample is not None:
        identity = self.downsample(x)

    out += identity # Shortcut connection
    out = self.relu(out)

    return out
```



Hyperparameter



- Learning rate (lr): [0.1, 0.01]
- Batch size:[32, 64]
- Epochs: 10
- Patience: 10
- Optimizer: SGD (Stochastic Gradient Descent)

Hasil

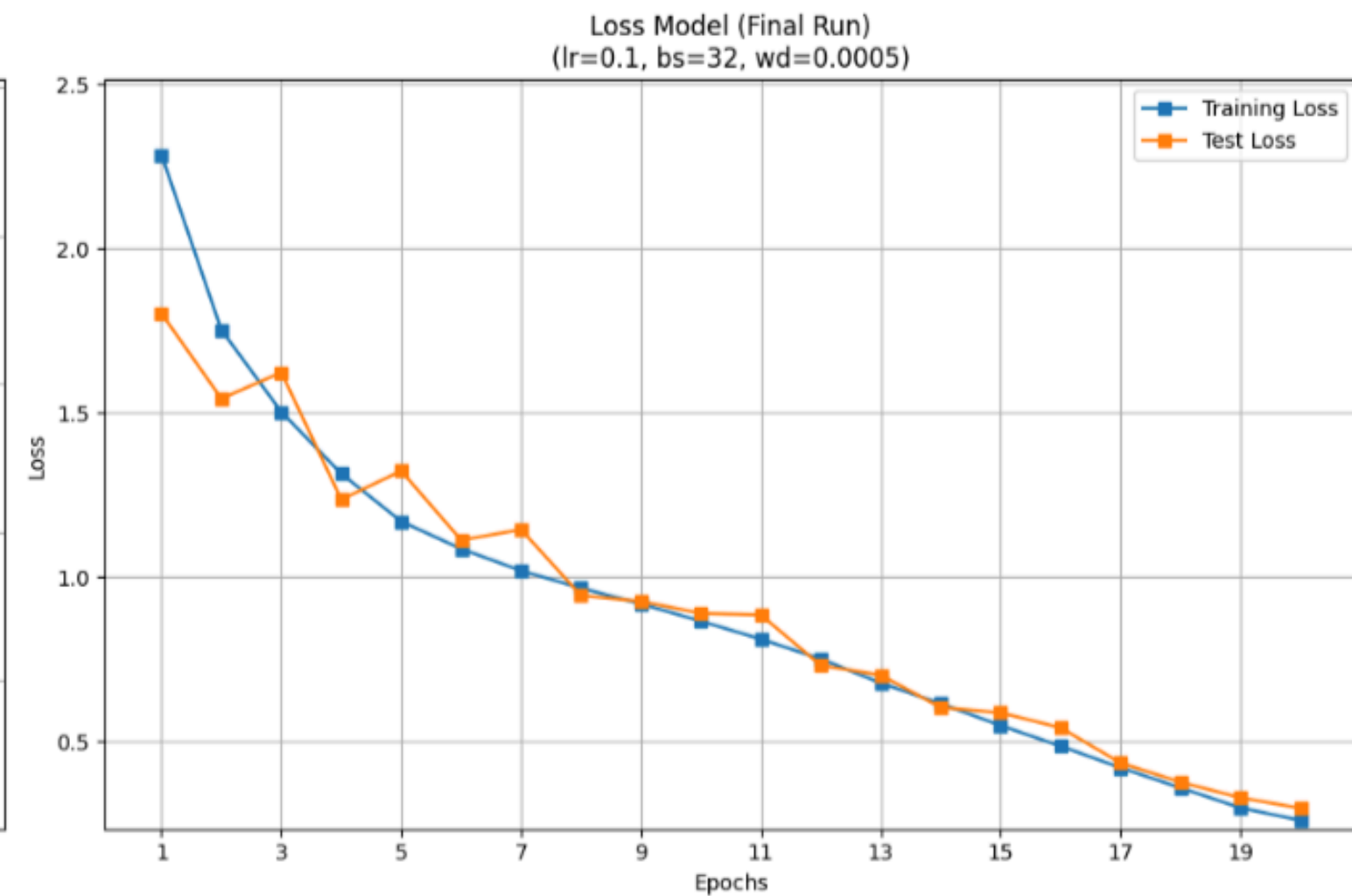
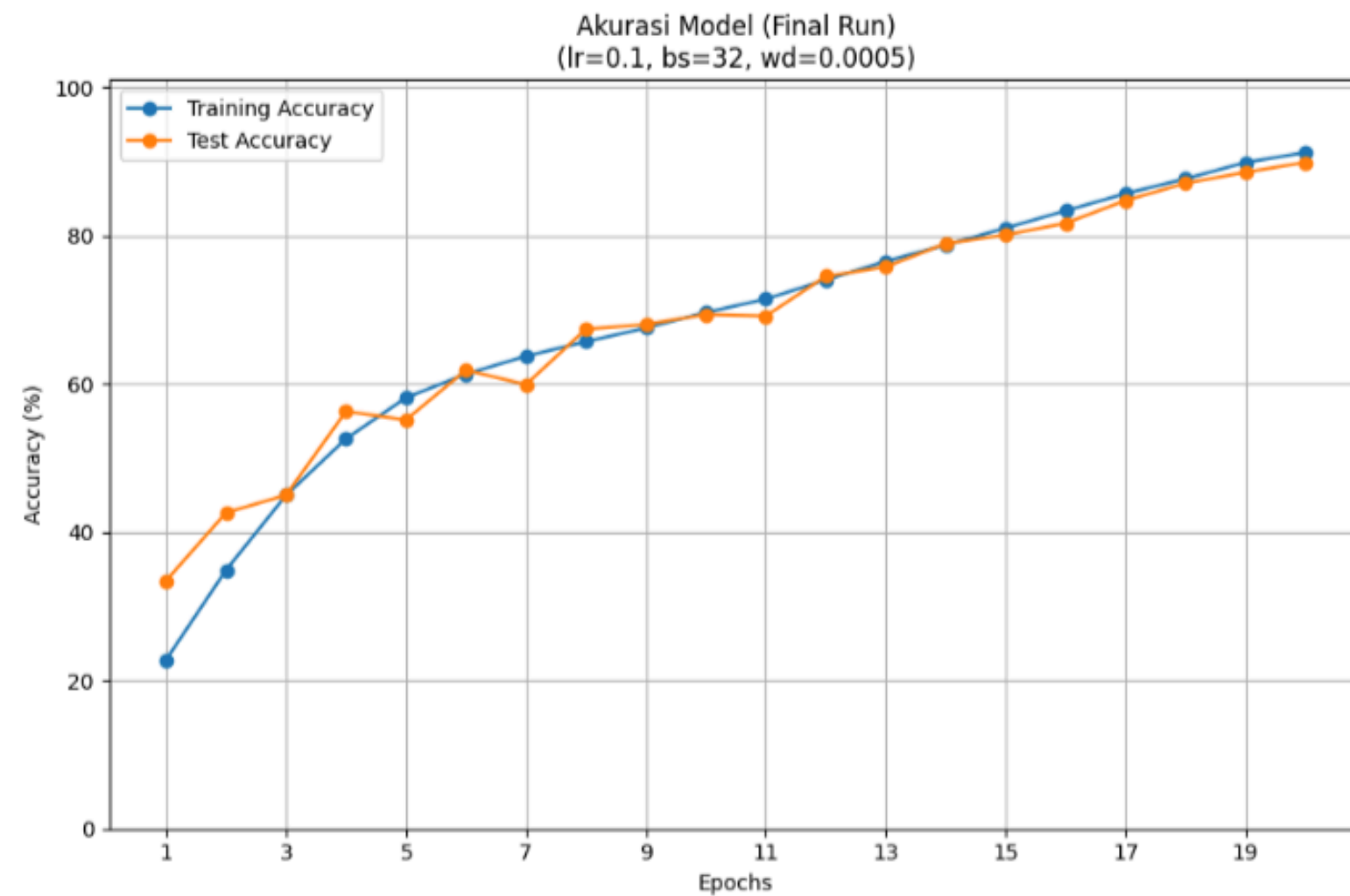
- lr terbaik= 0.1,
- Batch size terbaik= 32
- Akurasi: 82.18%



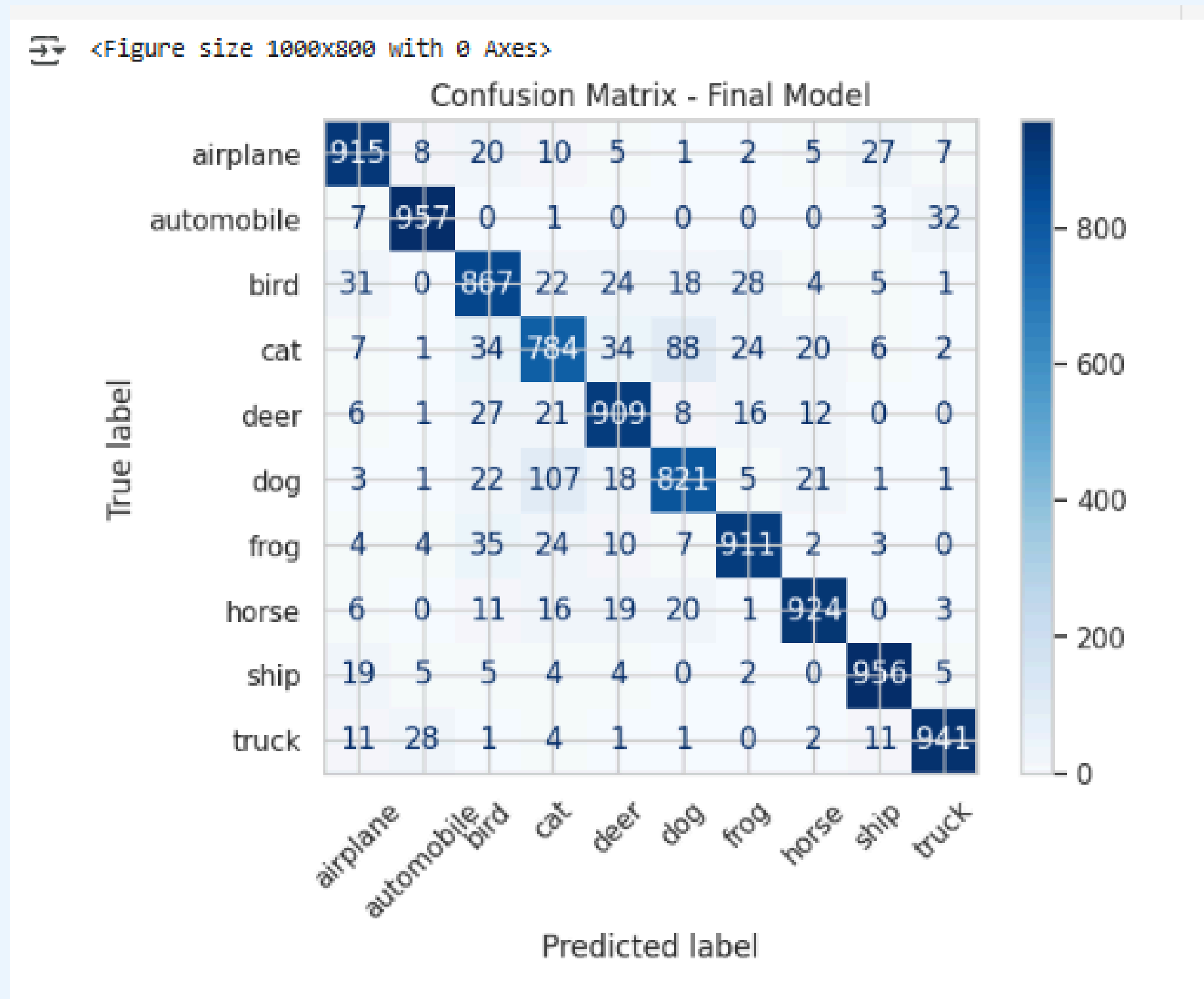
Grafik Loss dan Akurasi



Metrik Training & Test (Final Run): ResNet-50 CIFAR-10 (Test Acc: 89.85%)



Confusion Matrix

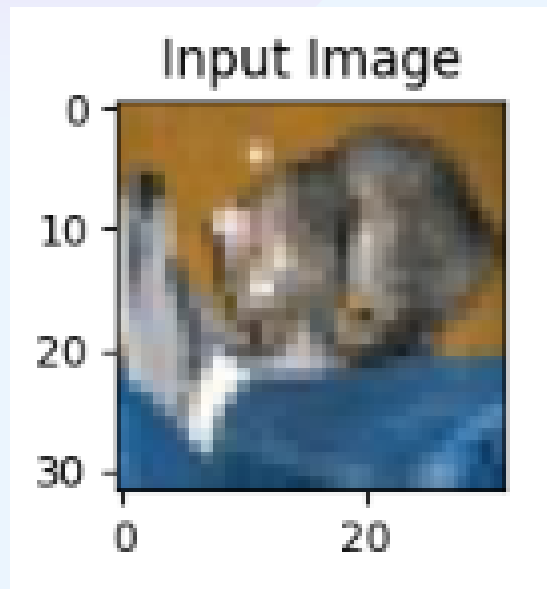


Laporan Klasifikasi Kelas

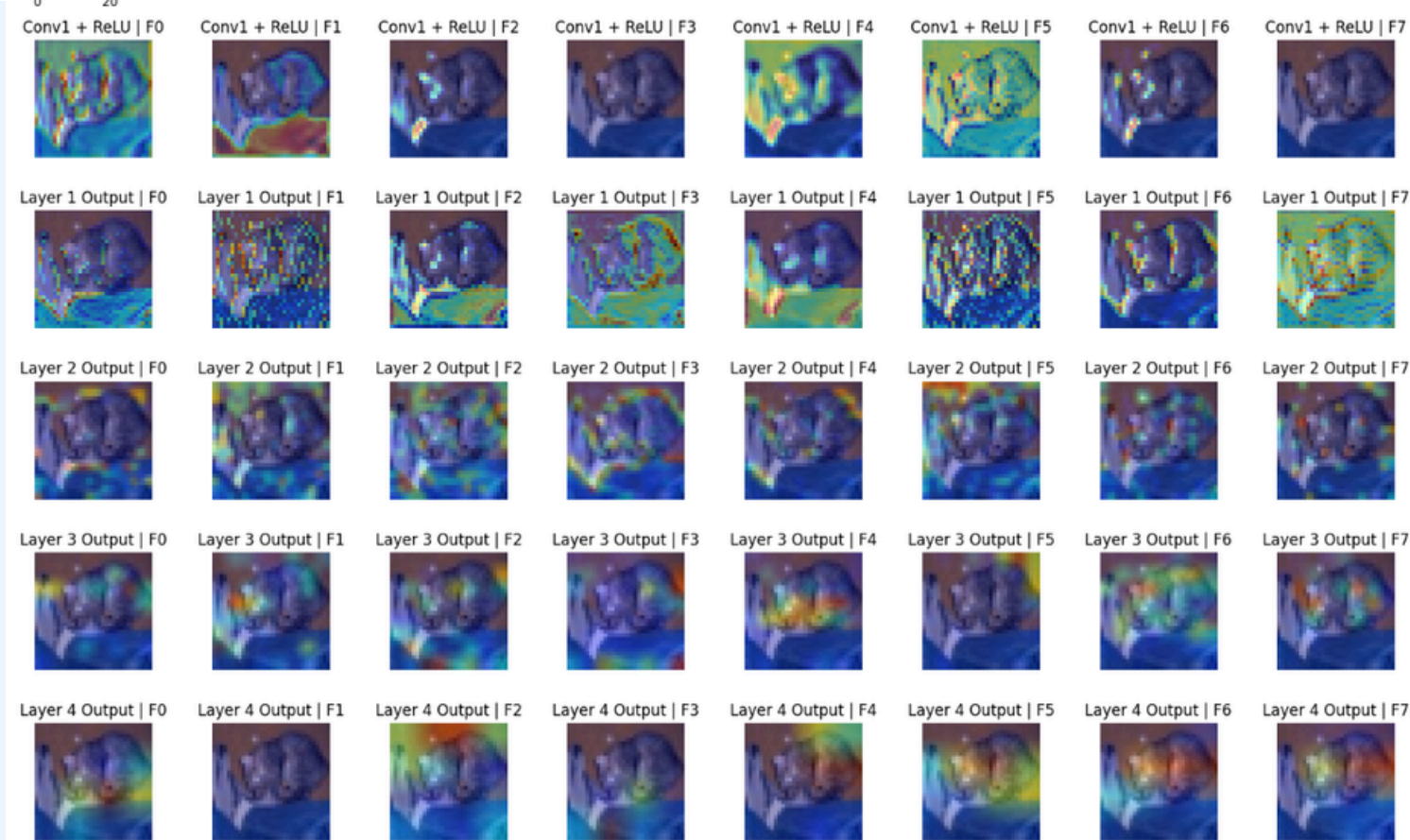


Kelas	Precision	Recall	F1 Score	
airplane	0.926	0.918	0.922	
automobile	0.946	0.957	0.951	
bird	0.931	0.867	0.898	
cat	0.885	0.784	0.831	
deer	0.933	0.909	0.921	
dog	0.919	0.821	0.867	
frog	0.947	0.911	0.929	
horse	0.933	0.924	0.928	
ship	0.958	0.956	0.957	
truck	0.943	0.941	0.942	

Feature Maps



Pred: Cat
True: Cat



1. Layer awal (Conv1 + ReLU dan Layer 1 Output)

- Layer awal belajar mengenali low-level features seperti: Edge, kontur, tekstur kasar
- Kontras antara bulu kucing dan background
- Feature maps di sini tajam dan bervariasi

2. Layer 2 Output

- Fitur menjadi lebih spesifik dan fokus ke bagian tubuh kucing seperti wajah, telinga, dan mata.
- Model mulai menggabungkan edge dan tekstur menjadi bentuk-bentuk yang bermakna.
- Beberapa channel terlihat mulai mengabaikan background

3. Layer 3 Output

- Fitur semakin semantik
- Feature map menunjukkan aktivasi yang konsisten pada area wajah/tubuh, walau dalam bentuk agak kabur.

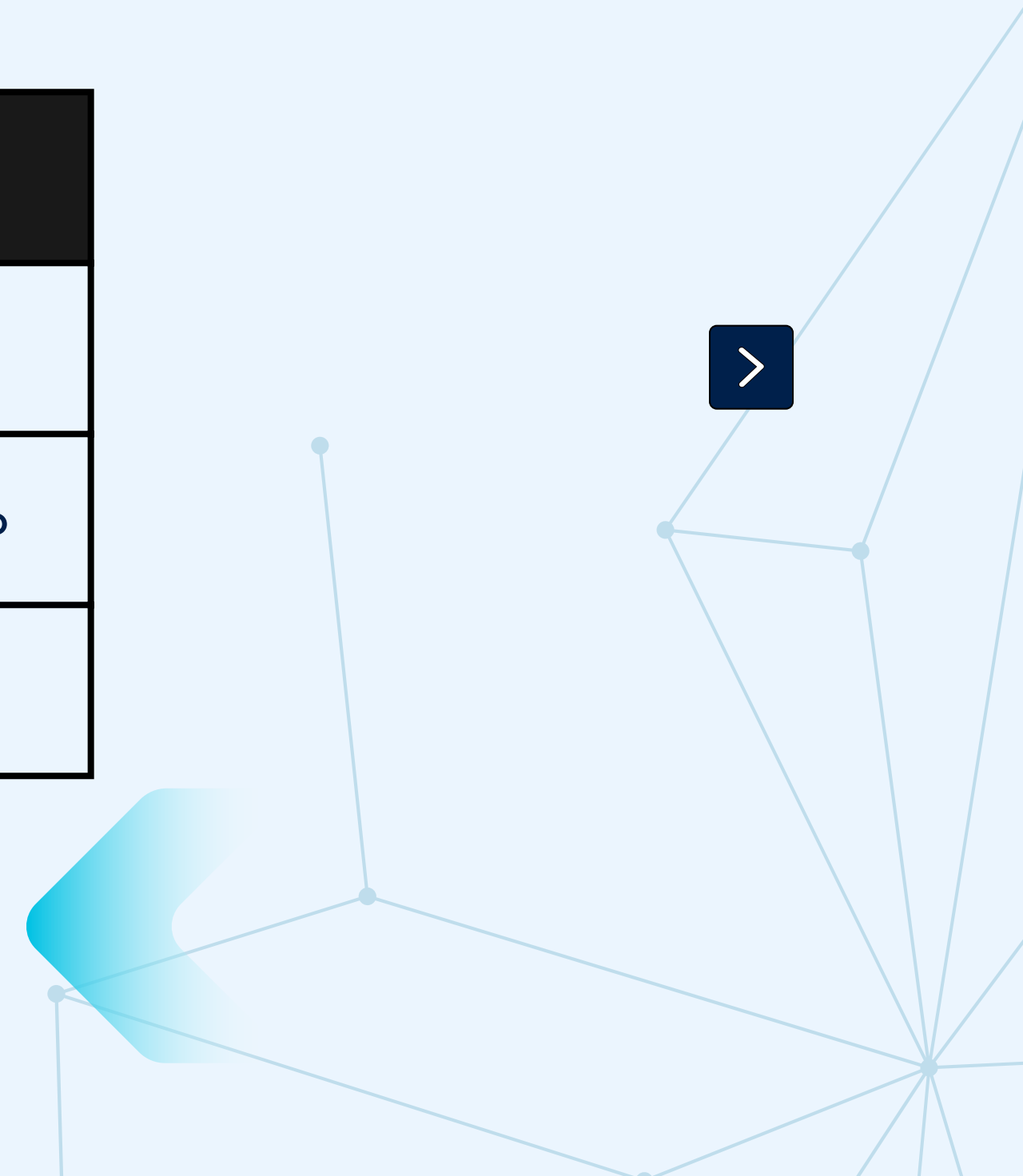
4. Layer 4 Output (deep layer)

- Wajah/tubuh terlihat sebagai satu objek utuh, bukan hanya bagian-bagian kecil.
- Di layer ini objek utama masih terlihat jelas, artinya ResNet berhasil mempertahankan informasi penting secara dalam.



Akurasi

	CNN	ResNet
Test Accuracy	79.86%	89.85%
Train Accuracy	80.66%	91.17%



Akurasi Kinerja Model

	CNN	ResNet
Test Accuracy	79.86%	89.85%
Precision	79.90%	93.21%
Recall	79.90%	89.88%
F1-Score	79.06%	91.46%



Akurasi per kelas

Label	CNN	ResNet
Plane	84.60%	91.80%
Car	90.50%	95.70%
Bird	69.90%	86.70%
Cat	52.40%	78.40%
Deer	84.30%	90.90%
Dog	68.80%	82.10%
Frog	89.60%	91.10%
House	83.40%	92.40%
Ship	85.20%	95.60%
Truck	89.90%	94.10%



Kesimpulan



Aspek	CNN	ResNet
Akurasi Test	79.86 %	89.85 %
Kedalaman Jaringan	Dangkal (8 convolutional layers)	Dalam (18 layer dengan residual block)
Kemampuan Generalisasi	Cukup baik, model stabil tanpa overfitting	Sangat baik, generalisasi tinggi ke data test
Overfitting	Tidak mengalami overfitting	Tidak tampak signifikan
Analisis Feature Map	Menangkap fitur dasar (tepi, warna, tekstur)	Menangkap fitur kompleks, semantik tinggi, fokus pada objek utama
Arsitektur	CNN konvensional dengan desain custom ringan	ResNet dengan skip connection dan struktur yang lebih dalam

The background is a dark blue gradient with faint, light blue geometric patterns. In the center, there is a faint, semi-transparent image of a person wearing a VR headset and holding controllers. To the left, there is a large, curved, metallic-looking object in shades of purple and blue. In the top left corner, there is a small, striped sphere. In the bottom right corner, there is a larger, ringed sphere, similar to Saturn, with a striped pattern. The text "THANK YOU!" is centered in a bold, white, sans-serif font.

THANK YOU!